

Code Smell & Refactoring

24가지 코드 냄새와 VSCode 리팩토링 기법

★ C++ (VSCode/CMake) 위주 · Java (JUnit 5/Maven) 참고 대응

C++17

VSCode

clangd

CMake Tools

Java 참고: Java 17+

Java 참고: Extension Pack for Java

Java 참고: Maven

- | | | | |
|---------------|--------------|-------------|-----------------|
| 1. 이해하기 힘든 이름 | 2. 중복 코드 | 3. 긴 함수 | 4. 긴 매개변수 목록 |
| 5. 전역 데이터 | 6. 가변 데이터 | 7. 뒤엎힌 변경 | 8. 산탄총 수술 |
| 9. 기능 편애 | 10. 데이터 뭉치 | 11. 기본형 집착 | 12. 반복되는 Switch |
| 13. 반복문 | 14. 성의 없는 요소 | 15. 추측성 일반화 | 16. 임시 필드 |
| 17. 메시지 체인 | 18. 중재자 | 19. 내부자 거래 | 20. 거대한 클래스 |
| 21. 대안 클래스들 | 22. 데이터 클래스 | 23. 상속 포기 | 24. 주석 |

리팩토링(Refactoring) 개요

외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 프로세스



설계 개선

중복 코드 제거 → 구조 개선



이해도 향상

코드를 빠르게 파악·수정 가능



버그 발견

단순한 구조 → 버그 원인 명확



개발 속도

깔끔한 설계 → 기능 추가 빠름

"리팩토링은 외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 방법으로, 소프트웨어 시스템을 변경하는 프로세스이다. 버그가 끼어들 가능성을 최소화하면서 코드를 정리하는 정형화된 방법이다."

1. 테스트 코드 작성

2. Code Smell 감지

3. 리팩토링 기법 선택

4. 리팩토링 수행

5. 테스트 재실행 (녹색 확인)

VSCode C++ 리팩토링 도구

- clangd (언어 서버): 이름 바꾸기, 자동 완성
- F2: 심볼 이름 바꾸기 (Rename Symbol)
- Ctrl+Shift+R: 리팩토링 메뉴
- Ctrl+.: Quick Fix (코드 액션)
- CMake Tools: 빌드·테스트 자동화
- C/C++ Extension Pack (Microsoft)

VSCode Java 리팩토링 도구

- Extension Pack for Java (Microsoft)
- F2: 이름 바꾸기 (Rename Symbol)
- Ctrl+Shift+R: 리팩토링 메뉴
- Ctrl+.: Quick Fix / Refactor
- Maven/Gradle: 빌드·테스트
- Java Language Support (Red Hat)

Refactoring 시 견지해 볼 수 있는 지침

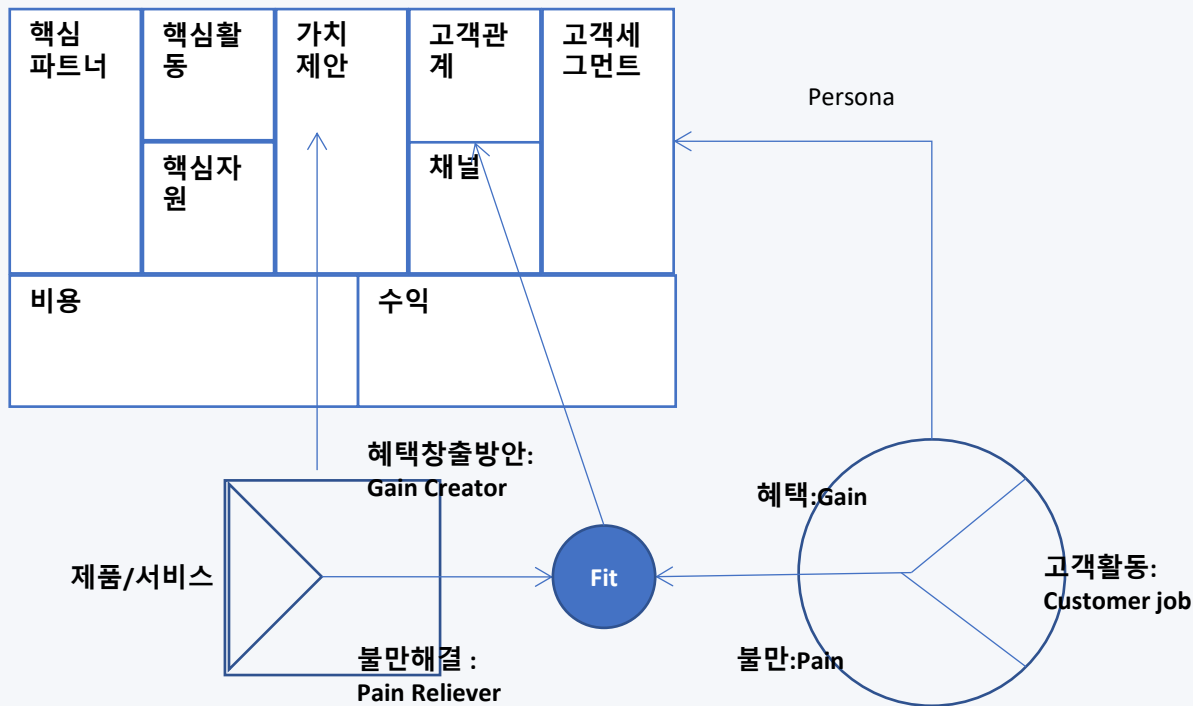
[켄트 벡] "두 개의 모자 비유(The Two Hats Metaphor)"

기능을 추가하는 것
리팩토링을 하는 것

구분하자

빠른 가치 제공

코드의 기술적 부채(Technical Debt) 해결



Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

- 여기, 사용자 데이터 포매팅 예 처리 순서

단계	코드	문제점	리팩토링 후 개선점
처음 (1번)	단일 메서드 <code>getUserFullName</code> 만 구현.	중복 없음.	개선할 필요 없음.
두 번째 (2번)	비슷한 메서드 <code>getAdminFullName</code> 추가.	코드 중복 발생 (이름 포매팅 로직이 여러 메서드에 중복).	아직은 리팩토링하지 않음.
세 번째 (3번)	비슷한 로직의 메서드가 3개 이상 반복.	중복된 코드는 유지보수성을 떨어뜨림. 이름 포매팅 규칙이 바뀔 경우, 모든 메서드를 수정해야 함.	<code>formatFullName</code> 메서드로 공통 로직을 추출하여 중복 제거.
더 나아간 리팩토링	이름 포매팅 로직을 <code>NameFormatter</code> 라는 유틸리티 클래스로 이동.	이름 포매팅 로직이 여러 클래스에서 중복될 가능성이 있음.	<code>NameFormatter</code> 를 사용하여 모든 이름 포매팅 로직이 재사용 가능해짐.

냄새 1. 이해하기 힘든 이름 (Mysterious Name)

리팩토링: 함수 선언 변경 · 변수/필드 이름 바꾸기

VSCode 단축키

C++: F2 (Rename Symbol) | Ctrl+Shift+R → Rename

Java: F2 (Rename Symbol) | Ctrl+Shift+R → Rename

C++ (VSCode / CMake)

```
// ❌ Before - 이름만 보서는 의도를 알 수 없음
int calc(int x, int y) { return x + y; }

struct U {
    std::string nm; // 무슨 뜻?
    int ag;
};

// ✅ After - 이름만으로 의도 전달
int calculateTotalPrice(int basePrice, int tax) {
    return basePrice + tax;
}

struct User {
    std::string name;
    int age;
};

// ✅ After - 변수 이름 바꾸기
void printSummary() {
    int firstNumber = 10;
    int secondNumber = 20;
    int result = firstNumber + secondNumber;
    std::cout << result << std::endl;
}

// VSCode: 이름에 커서 → F2 → 새 이름 입력 → Enter
// clangd가 모든 참조를 일괄 변경
```

Java 대응 (VSCode / Maven)

```
// ❌ Before
public int calc(int x, int y) {
    return x + y;
}

class U {
    String nm;
    int ag;
}

// ✅ After
public int calculateTotalPrice(
    int basePrice, int tax) {
    return basePrice + tax;
}

class User {
    String name;
    int age;
}

// ✅ 변수 이름 바꾸기
public void calculate() {
    int firstNumber = 10;
    int secondNumber = 20;
    int result = firstNumber
        + secondNumber;
    System.out.println(result);
}

// F2 → 새 이름 → Enter
// 모든 참조 일괄 변경
```

좋은 이름 찾기 팁: 주석으로 먼저 설명 작성 → 그 주석을 이름으로 변환 | clangd는 헤더·소스 파일의 모든 선언/정의를 한 번에 변경

리팩토링: 함수 추출 · 함수 올리기 · 메서드 통합

VSCode 단축키

C++: Ctrl+Shift+R → Extract Function | 선택 후 Ctrl+. → Extract

Java: Ctrl+Shift+R → Extract Method | Ctrl+. → Extract Method

C++ (VSCode / CMake)

```
// ❌ Before - 동일 로직이 두 곳에 중복
class Rectangle {
    double width, height;
public:
    double getArea() { return width * height; }
    double getScale() { return width * height * 2; }
    // width * height 가 중복!
};

class Triangle {
    double base, height;
public:
    double area() { return 0.5 * base * height; }
    double info() { return 0.5 * base * height; } // 중복!
};

// ✅ After - 함수 추출하기 (Extract Function)
class Rectangle {
    double width, height;
    double computeArea() const { return width * height; } // 추출
public:
    double getArea() { return computeArea(); }
    double getScale() { return computeArea() * 2; }
};

// ✅ 클래스 계층에서 중복 - 함수 올리기 (Pull Up Method)
class Shape {
protected:
    virtual double area() const = 0;
public:
    void printArea() const { // 공통 → 부모로 올리기
        std::cout << "Area: " << area() << std::endl;
    }
};

class Rect : public Shape { double area() const override { return w*h; } };
class Tri : public Shape { double area() const override { return 0.5*b*h; } };
```

Java 대응 (VSCode / Maven)

```
// ❌ Before - 중복 로직
class Rectangle {
    double w, h;
    double getArea() { return w * h; }
    double getScale() { return w * h * 2; } // 중복
}

// ✅ After - Extract Method
class Rectangle {
    double w, h;
    private double computeArea() {
        return w * h; // 추출
    }
    double getArea() { return computeArea(); }
    double getScale() { return computeArea()*2; }
}

// ✅ Pull Up Method - 부모로 올리기
abstract class Shape {
    abstract double area();
    void printArea() { // 공통 → 부모
        System.out.println(area());
    }
}

class Rect extends Shape {
    double area() { return w * h; }
}

// Ctrl+Shift+R → Extract Method
// 선택 영역 → Quick Fix → Extract
```

3. 냄새 3. 긴 함수 (Long Function)

리팩토링: 함수 추출 · 임시변수 질의로 교체 · 매개변수 객체 · 명령 교체

VSCode 단축키

C++: 선택 → Ctrl+Shift+R → Extract Function to ... | Ctrl+. → Extract

Java: 선택 → Ctrl+Shift+R → Extract Method | Ctrl+. → Extract Method

C++ (VSCode / CMake)

```
// ❌ Before - 모든 로직이 한 함수에
void processOrder(Order& order) {
    // 유효성 검사
    if (order.items.empty()) return;
    if (order.customerId < 0) return;
    // 금액 계산
    double total = 0;
    for (auto& item : order.items)
        total += item.price * item.qty;
    double discount = total > 100 ? total*0.1 : 0;
    total -= discount;
    // 결제 처리
    PaymentGateway pg;
    pg.charge(order.customerId, total);
    // 알림 발송
    EmailService es;
    es.sendConfirmation(order.customerId, total);
}

// ✅ After - 함수 추출 (Extract Function)
bool isValidOrder(const Order& o) {
    return !o.items.empty() && o.customerId >= 0;
}

double calculateTotal(const Order& o) {
    double total = 0;
    for (auto& item : o.items) total += item.price * item.qty;
    double discount = total > 100 ? total * 0.1 : 0;
    return total - discount;
}

void chargePayment(int customerId, double total) {
    PaymentGateway{}.charge(customerId, total);
}

void sendNotification(int customerId, double total) {
    EmailService{}.sendConfirmation(customerId, total);
}

void processOrder(Order& order) {
    if (!isValidOrder(order)) return;
    double total = calculateTotal(order);
    chargePayment(order.customerId, total);
    sendNotification(order.customerId, total);
}
```

Java 대응 (VSCode / Maven)

```
// ❌ Before - 긴 함수
void processOrder(Order order){
    if(order.items.isEmpty()) return;
    if(order.customerId < 0) return;
    double total = 0;
    for(Item item : order.items)
        total += item.price * item.qty;
    double discount =
        total>100 ? total*0.1 : 0;
    total -= discount;
    new PaymentGateway()
        .charge(order.customerId, total);
    new EmailService()
        .sendConfirm(order.customerId,total);
}

// ✅ After - Extract Method
private boolean isValid(Order o){
    return !o.items.isEmpty()
        && o.customerId >= 0;
}

private double calcTotal(Order o){
    double t = o.items.stream()
        .mapToDouble(i->i.price*i.qty)
        .sum();
    return t - (t>100 ? t*0.1 : 0);
}

void processOrder(Order order){
    if(!isValid(order)) return;
    double t = calcTotal(order);
    new PaymentGateway().charge(
        order.customerId, t);
}
```

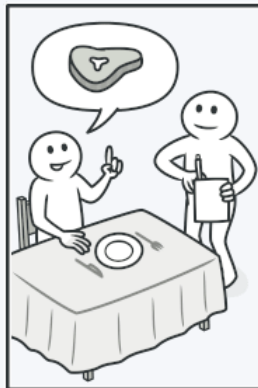
3. 냄새 3. 긴 함수 (Long Function)

함수를 명령으로 바꾸기(Replace Function with Command)

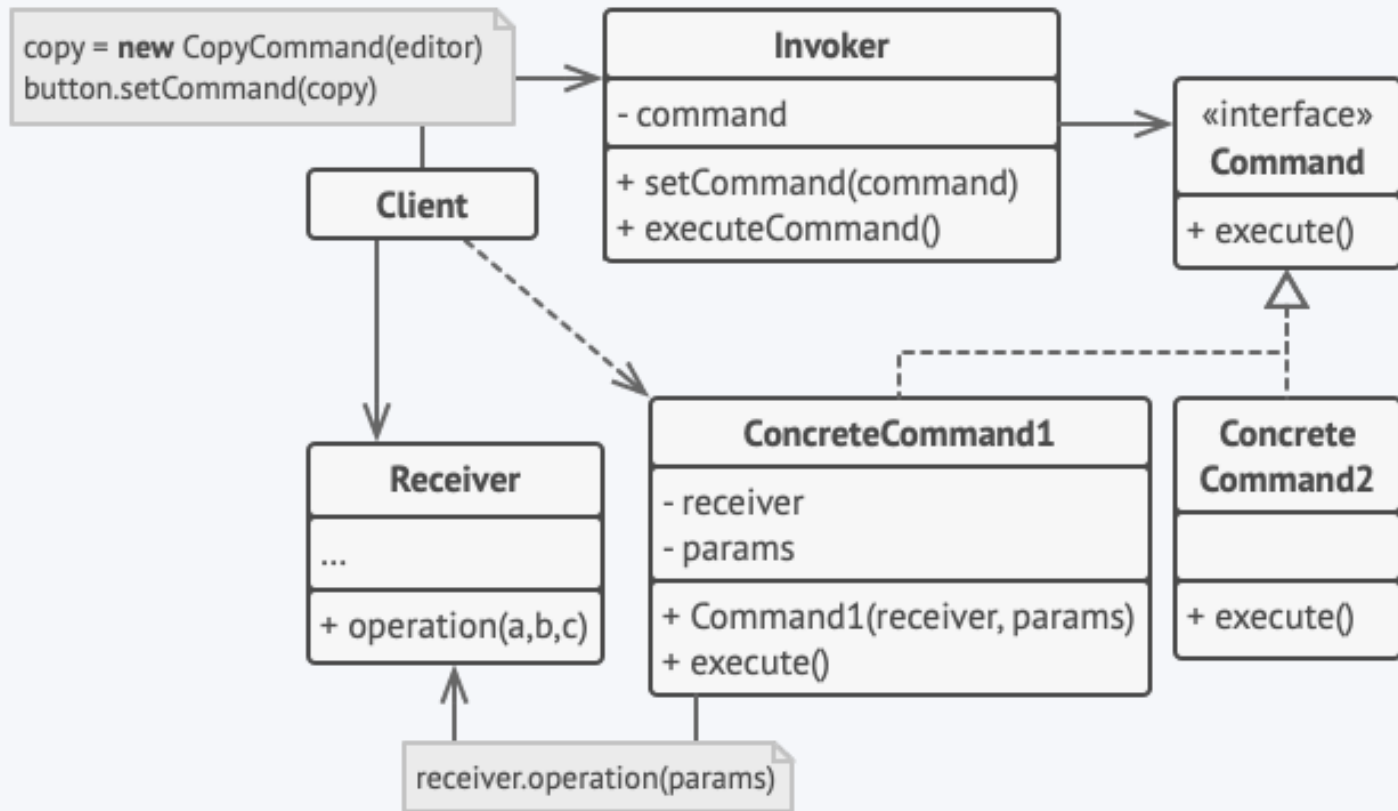
- Command 패턴이란?
 - Command 패턴은 행동(요청)을 객체로 캡슐화하는 디자인 패턴입니다.
 - 이를 통해 요청을 매개변수화, 저장, 큐잉, 로깅할 수 있으며, 호출자(Invoker)와 수신자(Receiver)를 분리합니다.
- 구성 요소:
 - Command: 실행할 작업을 캡슐화하는 객체 (예: TotalPriceCalculator)
 - Invoker: Command 객체를 호출하는 역할 (예: Order 클래스에서 TotalPriceCalculator 호출)
 - Receiver: 실제 작업을 수행하는 객체 (예: TotalPriceCalculator가 작업 수행)

요리사에게 주문이 전달되고 요리사는 주문을 읽고 그에 따라 음식을 요리합니다. 요리사는 주문과 함께 식사를 트레이에 놓습니다. 웨이터는 트레이를 발견한 후 당신이 주문한 대로 식사가 요리되었는지 확인하고 완성된 주문을 당신의 테이블로 가져옵니다.

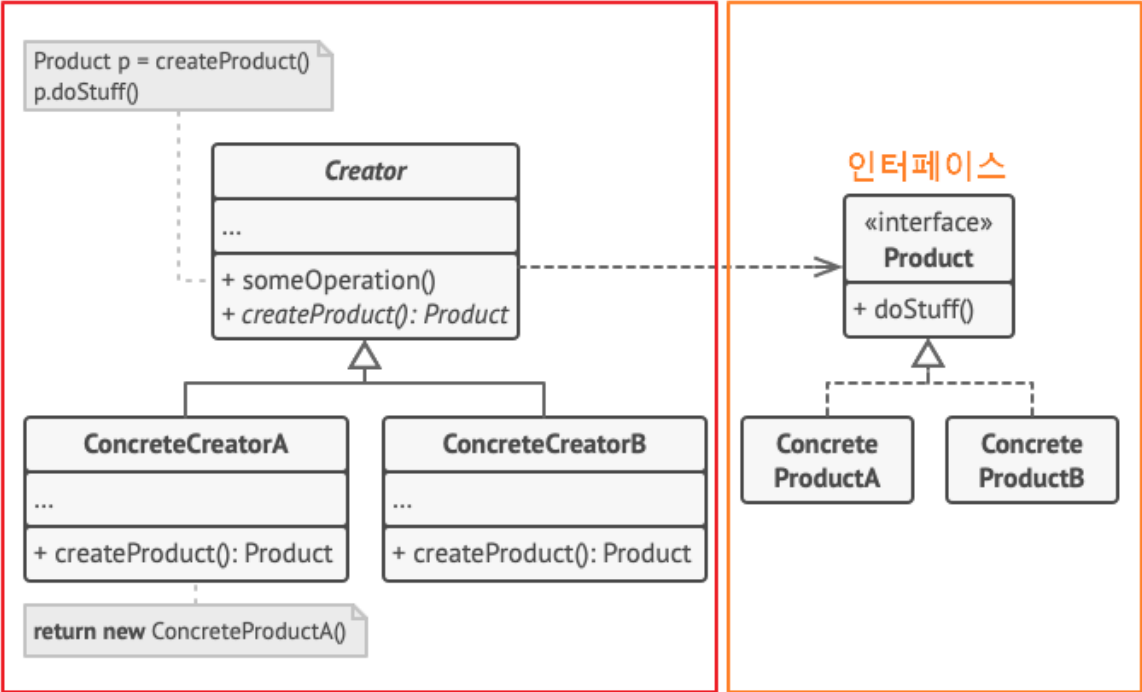
종이에 적힌 주문은 커맨드 역할을 합니다.



Command 패턴



Factory Method Pattern



공장 객체

→ 생성

제품 객체

냄새 4. 긴 매개변수 목록 (Long Parameter List)

리팩토링: 매개변수 객체 만들기 · 객체 통째로 넘기기 · 질의로 교체

VSCode 단축키

C++: Ctrl+Shift+R → Introduce Parameter Object (clangd)

Java: Ctrl+Shift+R → Introduce Parameter Object

C++ (VSCode / CMake)

```
// ❌ Before - 매개변수 4개 이상은 위험 신호
void createOrder(std::string customerName,
                 std::string address,
                 std::string city,
                 std::string zipCode,
                 double price,
                 int quantity,
                 std::string paymentMethod) { ... }

// ✅ After 1: 매개변수 객체 만들기
struct Address {
    std::string street, city, zipCode;
};
struct OrderInfo {
    std::string customerName;
    Address address;
    double price;
    int quantity;
    std::string paymentMethod;
};
void createOrder(const OrderInfo& info) { ... }

// ✅ After 2: 객체 통째로 넘기기 (Preserve Whole Object)
// Before:
void setRange(int low, int high) { ... }
int low = range.getLow();
int high = range.getHigh();
setRange(low, high);

// After:
void setRange(const Range& range) {
    // low = range.getLow(), high = range.getHigh() 직접 사용
}
setRange(range); // 객체를 통째로 전달

// ✅ After 3: 매개변수를 질의 함수로 바꾸기
// Before: double discount(double price, double taxRate)
// After: double discount(double price)
// { return price * getTaxRate(); }
```

Java 대응 (VSCode / Maven)

```
// ❌ Before
void createOrder(String name,
                 String address, String city,
                 String zip, double price,
                 int qty, String payment) {...}

// ✅ After 1: 매개변수 객체
record Address(String street,
               String city, String zip) {}
record OrderInfo(String name,
                 Address addr, double price,
                 int qty, String payment) {}

void createOrder(OrderInfo info) {...}

// ✅ After 2: 객체 통째로
// Before:
void setRange(int low, int high){}
// After:
void setRange(Range range) {
    // range.getLow() 직접 사용
}

// ✅ After 3: 질의로 교체
// Before:
// double discount(double p, double rate)
// After:
double discount(double price) {
    return price * getTaxRate();
}

// Ctrl+Shift+R →
// Introduce Parameter Object
```

매개변수 4개 이상 → 반드시 검토! | C++ struct/Java record로 매개변수 묶기 | 매개변수가 같이 다니면 → 데이터 뭉치(냄새 10)와 함께 확인

리팩토링: 변수 캡슐화하기 · 변수 쪼개기 · 질의/변경 함수 분리 · const/final 사용

C++ (VSCode / CMake)

```
// ❌ 냄새 5 - 전역 데이터
int g_currentTemperature = 0; // 전역 → 어디서든 변경 가능!
void setTemp(int t) { g_currentTemperature = t; }

// ✅ After - 변수 캡슐화하기 (Encapsulate Variable)
class TemperatureManager {
    int temperature_ = 0;
public:
    int getTemperature() const { return temperature_; }
    void setTemperature(int t) { temperature_ = t; }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 5 - 전역
static int currentTemp = 0;

// ✅ After - 캡슐화
class TemperatureManager {
    private int temperature = 0;
    public int get() { return temperature; }
    public void set(int t) { temperature=t; }
}
```

리팩토링: 변수 캡슐화하기 · 변수 쪼개기 · 질의/변경 함수 분리 · const/final 사용

C++ (VSCode / CMake)

```
// ❌ 캡새 6 - 가변 데이터
struct Point { int x, y; };
Point p{0, 0};
p.x = 10; // 어디서든 변경 가능 → 추적 어려움

// ✅ After 1 - 불변 값 객체 (const)
struct ImmutablePoint {
    const int x, y;
    ImmutablePoint(int x, int y) : x(x), y(y) {}
};
// 변경 대신 새 객체 생성

// ✅ After 2 - 변수 쪼개기 (Split Variable): 용도별 다른 변수 사용
// Before:
// double temp = 2 * (height + width); // 둘레
// temp = height * width;              // 면적 (재사용 → 혼란!)
// After:
double perimeter = 2 * (height + width);
double area      = height * width;

// ✅ After 3 - 질의/변경 함수 분리
// 읽기와 쓰기를 항상 분리!
int  getTotalCount() const { return count_; } // 질의만
void increment()         { ++count_; }       // 변경만
```

Java 대응 (VSCode / Maven)

```
// ❌ 캡새 6 - 가변
class Point { int x, y; }
Point p = new Point();
p.x = 10; // 어디서든 변경

// ✅ After 1 - 불변 값 객체
record Point(int x, int y) {}
// 불변 → 변경 시 새 객체 생성

// ✅ After 2 - 변수 쪼개기
// ❌ Before
// double tmp = 2*(h+w); // 둘레
// tmp = h * w;          // 면적 재사용
// ✅ After
double perimeter = 2*(h+w);
double area      = h * w;

// ✅ After 3 - 질의/변경 분리
int  getTotalCount() { return count; }
void increment()     { count++; }

// Java: final 필드로 불변성 보장
class Config {
    private final String url;
    Config(String url){ this.url=url;}
}
```

리팩토링: 단계 쪼개기 · 함수/필드 옮기기 · 클래스 추출/인라인

7. 뒤틀린 변경

하나의 클래스가 여러 이유로 수정됨 → SRP 위반

해결: 클래스 추출하기 (Extract Class)

C++ (VSCode / CMake)

```
// ❌ 냄새 7 - 하나의 클래스가 DB도 하고 가격 계산도
class Order {
    void saveToDatabase() { ... } // DB 역할
    void loadFromDatabase() { ... }
    double calculateDiscount() { ... } // 가격 계산 역할
    double applyTax(double price) { ... }
};
// DB 변경할 때도, 가격 정책 바꿀 때도 이 클래스 수정!

// ✅ After - 단계 쪼개기 / 클래스 추출
class OrderRepository { // DB 전담
    void save(const Order& o) { ... }
    Order load(int id) { ... }
};
class PricingEngine { // 가격 전담
    double discount(const Order& o) { ... }
    double applyTax(double price) { ... }
};
::calculate(amount); } };
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 7 - SRP 위반
class Order {
    void saveToDB() { ... } // DB
    void loadFromDB() { ... }
    double discount() { ... } // 가격
    double applyTax() { ... }
}

// ✅ After - 클래스 추출
class OrderRepository {
    void save(Order o) { ... }
    Order load(int id) { ... }
}
class PricingEngine {
    double discount(Order o) { ... }
    double applyTax(double p) { ... }
}
```

리팩토링: 단계 쪼개기 · 함수/필드 옮기기 · 클래스 추출/인라인

8. 산탄총 수술

하나의 변경이 여러 클래스에 영향 → 응집도 부족

해결: 함수/필드 옮기기 (Move Function/Field)

C++ (VSCode / CMake)

```
// ❌ 냄새 8 - 세금 정책 변경 시 여러 클래스 수정 필요
class Order { double tax() { return price * 0.1; } };
class Receipt { double tax() { return amount * 0.1; } };
class Invoice { double tax() { return total * 0.1; } };
// 세율 0.1 → 0.12로 바꾸면? 3곳 모두 수정!

// ✅ After - 함수 옮기기 (Move Function)
class TaxCalculator {
    static double calculate(double amount) {
        return amount * TAX_RATE;
    }
    static constexpr double TAX_RATE = 0.1;
};
class Order { double tax() { return TaxCalculator::calculate(price); } };
class Receipt { double tax() { return TaxCalculator::calculate(amount); } };
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 8 - 산탄총 수술
class Order { double tax(){return p*0.1;}}
class Receipt{ double tax(){return a*0.1;}}
class Invoice{ double tax(){return t*0.1;}}
// 세율 바꾸면 3곳 모두 수정!

// ✅ After - Move Function
class TaxCalc {
    static final double RATE = 0.1;
    static double calc(double amount){
        return amount * RATE;
    }
}
class Order{
    double tax(){return TaxCalc.calc(p);}
}
```

9. 냄새 9. 기능 편애 (Feature Envy)

리팩토링: 함수 옮기기 (Move Function) · 전략 패턴 적용

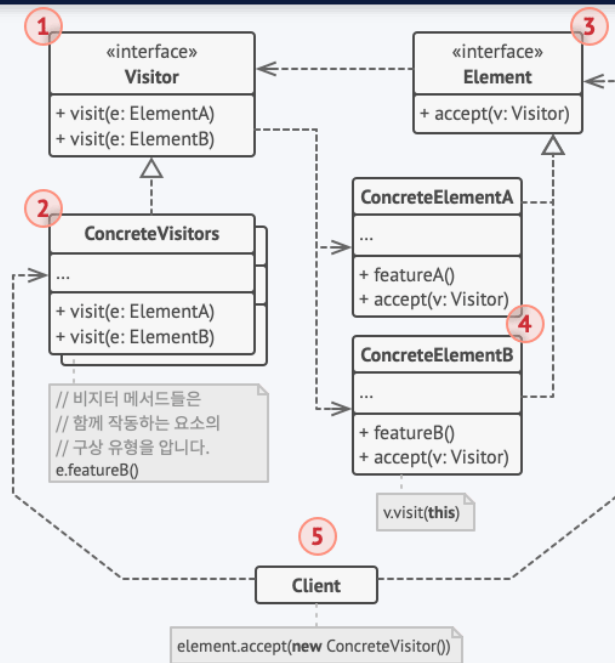
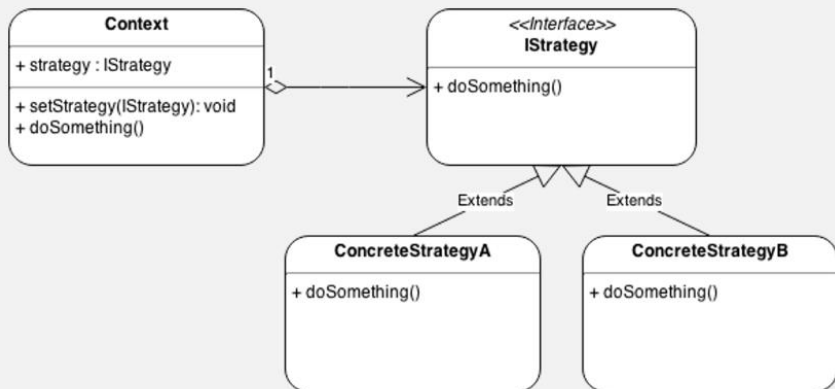
VSCode 단축키

C++: 함수 선택 → Ctrl+Shift+R → Move to ... | Ctrl+. → Move Function

Java: 함수 선택 → Ctrl+Shift+R → Move to ... | Ctrl+. → Move Method

판별법: 한 메서드가 자신이 속한 클래스보다 다른 클래스의 데이터를 더 많이 사용하면 → 그 클래스로 이동! | 전략 패턴/방문자 패턴으로도 해결 가능

Strategy pattern – Class diagram



9. 냄새 9. 기능 편애 (Feature Envy)

C++ (VSCode / CMake)

```
// ❌ Before - Invoice가 Address 데이터를 과도하게 사용 (기능 편애)
struct Address {
    std::string street, city, zipCode;
};
struct Customer {
    std::string name;
    Address address;
};
class Invoice {
public:
    void printCustomerInfo(const Customer& c) {
        // Address의 데이터를 Invoice가 직접 조합 → 기능 편애!
        std::cout << c.address.street << ", "
                    << c.address.city << " "
                    << c.address.zipCode << std::endl;
    }
};

// ✅ After - formatAddress() 를 Address로 이동 (Move Function)
struct Address {
    std::string street, city, zipCode;
    // 책임이 Address에 있는 것이 자연스러움
    std::string format() const {
        return street + ", " + city + " " + zipCode;
    }
};
class Invoice {
public:
    void printCustomerInfo(const Customer& c) {
        std::cout << c.address.format() << std::endl;
        // Invoice는 단순히 메시지만 전달
    }
};

// VSCode: 함수 이름 선택 → Ctrl+Shift+R → Move to ...
```

Java 대응 (VSCode / Maven)

```
// ❌ Before - 기능 편애
class Address {
    String street, city, zipCode;
}
class Invoice {
    void printInfo(Customer c) {
        // Address 데이터를 Invoice가 직접 다룸!
        Address a = c.getAddress();
        System.out.println(
            a.getStreet() + ", " +
            a.getCity() + " " +
            a.getZipCode());
    }
}

// ✅ After - formatAddress() 이동
class Address {
    String street, city, zipCode;
    // 책임을 Address로 이동
    public String format() {
        return street + ", " +
            city + " " + zipCode;
    }
}
class Invoice {
    void printInfo(Customer c) {
        // 단순 메시지 전달만
        System.out.println(
            c.getAddress().format());
    }
}

// Ctrl+. → Move Method
```

10. 냄새 10. 데이터 뭉치 (Data Clumps)

리팩토링: 클래스 추출 · 기본형을 객체로 바꾸기 · 타입 코드를 서브클래스로

C++ (VSCode / CMake)

```
// ❌ 냄새 10 - 항상 같이 다니는 데이터 → 클래스로 묶기
// Before: 전화번호 3개가 항상 함께 전달
void print(const std::string& areaCode,
           const std::string& prefix,
           const std::string& number) { ... }

// ✅ After - 클래스 추출하기 (Extract Class)
struct PhoneNumber {
    std::string areaCode, prefix, number;
    std::string format() const {
        return "(" + areaCode + ") " + prefix + "-" + number;
    }
};

void print(const PhoneNumber& phone) {
    std::cout << phone.format();
}
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 10 - 데이터 뭉치
void print(String areaCode,
           String prefix, String number){}

// ✅ After - Extract Class
record PhoneNumber(
    String areaCode,
    String prefix,
    String number) {
    String format(){
        return "("+areaCode+") "
            +prefix+"-"+number;
    }
}

void print(PhoneNumber p){
    System.out.println(p.format());
}
```

리팩토링: 클래스 추출 · 기본형을 객체로 바꾸기 · 타입 코드를 서브클래스로

C++ (VSCode / CMake)

```
// ❌ 냄새 11 - 기본형으로 의미 있는 개념 표현
double price = 19.99; // 그냥 double - 음수 방지 로직 어디?

// ✅ After 1 - 기본형을 객체로 바꾸기 (Replace Primitive with Object)
class Price {
    double value_;
public:
    explicit Price(double v) {
        if (v < 0) throw std::invalid_argument("Price cannot be negative");
        value_ = v;
    }
    double value() const { return value_; }
    std::string toString() const {
        return "$" + std::to_string(value_);
    }
};

// ✅ After 2 - 타입 코드를 서브클래스로 바꾸기
// Before: int type = 1(Engineer)/2(Manager)
// After:
class Employee { public: virtual double bonus(double sal) const = 0; };
class Engineer : public Employee { double bonus(double s) const override {
return s*0.1; } };
class Manager : public Employee { double bonus(double s) const override {
return s*0.2; } };
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 11 - 기본형 집착
double price = 19.99;

// ✅ After 1 - 기본형을 객체로
class Price {
    private final double value;
    Price(double v){
        if(v<0) throw new
            IllegalArgumentException();
        value = v;
    }
    double getValue(){ return value; }
    public String toString(){
        return String.format("%.2f",value);
    }
}

// ✅ After 2 - 타입 코드를 서브클래스로
abstract class Employee{
    abstract double bonus(double sal);
}
class Engineer extends Employee{
    double bonus(double s){return s*0.1;}
}
```

리팩토링: 조건부 로직을 다형성으로, 반복문을 파이프라인으로

C++ (VSCode / CMake)

```
// ❌ 냄새 12 - switch가 여러 곳에 반복
// 새 타입 추가 시 모든 switch 수정 필요!
double calcBonus(EmployeeType t, double sal) {
    switch(t) {
        case ENGINEER: return sal * 0.1;
        case MANAGER: return sal * 0.2;
        default: throw std::invalid_argument("unknown");
    }
}

std::string describe(EmployeeType t) {
    switch(t) { // 또 switch!
        case ENGINEER: return "Engineer";
        case MANAGER: return "Manager";
    }
}

// ✅ After - 다형성으로 교체 (Replace Conditional with Polymorphism)
class Employee {
public:
    virtual double bonus(double sal) const = 0;
    virtual std::string describe() const = 0;
    virtual ~Employee() = default;
};

class Engineer : public Employee {
public:
    double bonus(double s) const override { return s * 0.1; }
    std::string describe() const override { return "Engineer"; }
};

class Manager : public Employee {
public:
    double bonus(double s) const override { return s * 0.2; }
    std::string describe() const override { return "Manager"; }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 12 - 반복 switch
double bonus(int type, double sal){
    switch(type){
        case 1: return sal*0.1;
        case 2: return sal*0.2;
        default: throw new ...;
    }
}

// 또 다른 곳에 동일 switch!

// ✅ After - 다형성
abstract class Employee {
    abstract double bonus(double sal);
    abstract String describe();
}

class Engineer extends Employee {
    double bonus(double s){return s*0.1;}
    String describe(){return "Engineer";}
}
```

Switch 1개는 OK — 반복되는 switch가 문제 | C++20
 ranges::views = Java Stream API에 대응 | std::views::filter =
 .filter() / std::views::transform = .map()

리팩토링: 조건부 로직을 다형성으로 · 반복문을 파이프라인으로

C++ (VSCode / CMake)

```
// ❌ 냄새 13 - 고전적 반복문
std::vector<std::string> names;
for (const auto& emp : employees)
    if (emp.office == "London")
        names.push_back(emp.name);

// ✅ After - 파이프라인으로 바꾸기 (C++20 ranges)
#include <ranges>
auto names = employees
    | std::views::filter([](const auto& e){ return e.office=="London"; })
    | std::views::transform([](const auto& e){ return e.name; });
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 13 - 고전 반복문
List<String> names = new ArrayList<>();
for(Employee e : employees)
    if("London".equals(e.office))
        names.add(e.name);

// ✅ After - Stream 파이프라인
List<String> names = employees.stream()
    .filter(e->"London".equals(e.office))
    .map(e -> e.name)
    .collect(Collectors.toList());

// Stream: filter / map / reduce
// → 무엇을 하는지 한눈에 파악
```

리팩토링: 클래스/함수 인라인 · 죽은 코드 제거 · 클래스 추출 · 특이 케이스

14. 성의 없는 요소

쓸모없이 단순한 클래스/함수 → Inline / 제거

C++ (VSCode / CMake)

```
// ❌ 냄새 14 - 너무 단순한 클래스 (가치 없음)
class Multiplier {
    int multiply(int a, int b) { return a * b; }
};
// ✅ After - 함수 인라인 (Inline Function), 클래스 제거
// 그냥 a * b 사용하거나 static 유틸로 이동
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 14 - 단순한 클래스
class Multiplier {
    int multiply(int a, int b) {
        return a * b;
    }
}
// ✅ After - 인라인 / 제거
// 그냥 a * b 직접 사용
```

리팩토링: 클래스/함수 인라인·죽은 코드 제거·클래스 추출·특이 케이스

15. 추측성 일반화

언젠가 쓸 것 같아 만든 불필요한 추상화 → 삭제

C++ (VSCode / CMake)

```
// ❌ 냄새 15 - 미래를 위해 만든 불필요한 추상화
class AbstractDataProcessor {
    virtual void process() = 0; // 구현체가 딱 하나뿐!
    virtual void preProcess() {} // 아무도 안 씀
    virtual void postProcess() {}
};
// ✅ After - 추상 클래스 제거, 구체 클래스 직접 사용
// 죽은 코드 제거 (Remove Dead Code)
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 14 - 단순한 클래스
class Multiplier {
    int multiply(int a, int b) {
        return a * b;
    }
}
// ✅ After - 인라인 / 제거
// 그냥 a * b 직접 사용
```

리팩토링: 클래스/함수 인라인 · 죽은 코드 제거 · 클래스 추출 · 특이 케이스

16. 임시 필드

특정 상황에만 값이 있는 필드 → 클래스 추출 / 특이 케이스

C++ (VSCode / CMake)

```
// ❌ 냄새 16 - 조건에 따라 의미 없는 필드
class Order {
    std::string customerName;
    std::string shippingAddress;
    // 인터넷 주문일 때만 유효 - 매장 주문이면 항상 empty!
    std::string trackingNumber;
};

// ✅ After - 특이 케이스 클래스 추출
class Order {
    std::string customerName;
    virtual bool requiresShipping() const = 0;
    virtual ~Order() = default;
};

class OnlineOrder : public Order {
    std::string shippingAddress;
    std::string trackingNumber; // 온라인 전용
    bool requiresShipping() const override { return true; }
};

class StoreOrder : public Order {
    bool requiresShipping() const override { return false; }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 16 - 임시 필드
class Order {
    String customerName;
    String trackingNumber; // 온라인만
    // 매장 주문이면 null → 혼란!
}

// ✅ After - Introduce Special Case
abstract class Order {
    String customerName;
    abstract boolean requiresShipping();
}

class OnlineOrder extends Order {
    String trackingNumber; // 전용
    boolean requiresShipping(){return true;}
}

class StoreOrder extends Order {
    boolean requiresShipping(){return false;}
}
```


17 냄새 17. 메시지 체인 (Message Chains)

리팩토링: 위임 숨기기·중재자 제거하기·서브클래스를 위임으로

C++ (VSCode / CMake)

```
// ❌ 냄새 17 - 긴 메시지 체인 (Law of Demeter 위반)
// 내부 구조에 강하게 결합!
std::string dept = company.getHead()
                        .getDepartment()
                        .getHead()
                        .getName();

// ✅ After - 위임 숨기기 (Hide Delegate)
// Company 클래스에 편의 메서드 추가
class Company {
    std::string getDepartmentHeadName() const {
        return head_.getDepartment().getHead().getName();
    }
};
// 사용처: company.getDepartmentHeadName();
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 17 - 메시지 체인
String name = company.getHead()
                    .getDepartment()
                    .getHead()
                    .getName(); // 구조에 강하게 결합

// ✅ After - Hide Delegate
class Company {
    String getDeptHeadName() {
        return head.getDepartment()
                    .getHead().getName();
    }
}
company.getDeptHeadName(); // 단순화
```

리팩토링: 위임 숨기기, 중재자 제거하기, 서브클래스를 위임으로

C++ (VSCode / CMake)

```
// ❌ 냄새 18 - 중재자 (모든 메서드가 위임만)
class Person {
    Department dept;
public:
    Manager getManager() { return dept.getManager(); } // 위임만
    int numReports() { return dept.numReports(); } // 위임만
    // Person이 하는 일이 없음!
};

// ✅ After - 중재자 제거 (Remove Middle Man)
// Person을 거치지 않고 dept에 직접 접근
// person.dept.getManager(); - 또는:
class Person {
    Department dept; // public으로 노출
};

// ✅ After - 상속 대신 위임으로 (Replace Superclass with Delegate)
// Before: AdvancedPrinter extends Printer (불필요한 상속)
class AdvancedPrinter {
    Printer printer_; // 위임!
public:
    explicit AdvancedPrinter(Printer p) : printer_(p) {}
    void print(const std::string& msg) {
        printer_.print(msg); // 위임
    }
    void printWithTimestamp(const std::string& msg) {
        auto ts = std::chrono::system_clock::now();
        std::cout << "[" << ts.time_since_epoch().count() << "]" " << msg;
    }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 18 - 모든 메서드가 위임만
class Person {
    Department dept;
    Manager getManager(){
        return dept.getManager(); // 위임만
    }
    int numReports(){
        return dept.numReports(); // 위임만
    }
}

// ✅ After - Remove Middle Man
// Person 제거 후 dept 직접 접근

// ✅ 서브클래스를 위임으로
// Before: class AdvPrinter extends Printer
// After:
class AdvancedPrinter {
    private Printer printer; // 위임!
    AdvancedPrinter(Printer p){printer=p;}
    void print(String msg){
        printer.print(msg); // 위임
    }
    void printWithTs(String msg){
        System.out.println(
            System.currentTimeMillis()
            + " " + msg);
    }
}
```

19 냄새 19. 내부자 거래 (Insider Trading)

리팩토링: 함수/필드 옮기기 · 클래스 추출 · 슈퍼클래스 추출

C++ (VSCode / CMake)

```
// ❌ 냄새 19 - 내부자 거래 (결합도 과도)
class Department {
    Employee boss_; // private
    friend class Payroll; // Payroll이 내부 직접 접근!
};

class Payroll {
    void process(Department& d) {
        // d.boss_ 직접 접근 → 강한 결합!
        std::cout << d.boss_.salary;
    }
};

// ✅ After - Hide Delegate: 인터페이스 제공
class Department {
    Employee boss_;
public:
    double getBossSalary() const { return boss_.salary; }
};

class Payroll {
    void process(const Department& d) {
        std::cout << d.getBossSalary(); // 공개 인터페이스 사용
    }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 19 - 내부자 거래
class Department {
    Employee boss; // private
}

class Payroll {
    void process(Department d) {
        // 리플렉션 등으로 직접 접근
        System.out.println(d.boss.salary);
    }
}

// ✅ After - 공개 인터페이스 제공
class Department {
    private Employee boss;
    public double getBossSalary(){
        return boss.salary;
    }
}

class Payroll {
    void process(Department d) {
        System.out.println(
            d.getBossSalary()); // OK
    }
}
```

20 냄새 20. 거대한 클래스 (Large Class)

리팩토링: 함수/필드 옮기기, 클래스 추출, 슈퍼클래스 추출

C++ (VSCode / CMake)

// ❌ 냄새 20 - 거대한 클래스 (너무 많은 책임)

```
class Person {
    std::string name, email;
    std::string street, city, zip;    // 주소 정보
    std::string cardNumber, expiry;   // 결제 정보
    std::string username, password;  // 인증 정보
    void validateEmail() { ... }
    void processPayment() { ... }
    void authenticate() { ... }
    // ... 수십 개의 메서드
};
```

// ✅ After - 클래스 추출 (Extract Class)

```
struct ContactInfo { std::string email; void validate() { ... } };
struct Address      { std::string street, city, zip; };
struct Payment      { std::string cardNumber, expiry; void process() { ... } };
struct Auth         { std::string username, password; bool authenticate() { ... } };
};
```

```
class Person {
    std::string name;
    ContactInfo contact;
    Address      address;
    Payment      payment;
    Auth         auth;
    // 각 책임이 분리됨
};
```

Java 대응 (VSCode / Maven)

// ❌ 냄새 20 - 거대한 클래스

```
class Person {
    String name, email;
    String street, city, zip; // 주소
    String cardNumber, expiry; // 결제
    String username, password; // 인증
    void validateEmail(){}
    void processPayment(){}
    void authenticate(){}
}
```

// ✅ After - Extract Class

```
record ContactInfo(String email){}
record Address(String st, String city){}
record Payment(String card, String exp){}
class Person {
    String name;
    ContactInfo contact;
    Address address;
    Payment payment;
}
```

20번 기준: 필드 7개 이상 or 메서드 10개 이상 → 추출 검토 | 접두사 붙은 필드 그룹(address*, payment*) → Extract Class 신호 | VSCode: Ctrl+Shift+R → Extract Class

리팩토링: 인터페이스 통일 · 함수 옮기기 · setter 제거 · 메서드 숨기기

C++ (VSCode / CMake)

```
// ❌ 냄새 21 - 같은 역할인데 인터페이스가 다름
class EmailSender {
    void sendEmail(const std::string& to, const std::string& body) { ... }
};
class SMSSender {
    void transmit(const std::string& num, const std::string& msg) { ... }
    // sendEmail vs transmit - 같은 역할, 다른 이름!
};

// ✅ After - 인터페이스 통일 (슈퍼클래스 추출)
class Notifier {
public:
    virtual void send(const std::string& recipient,
                     const std::string& message) = 0;
    virtual ~Notifier() = default;
};
class EmailNotifier : public Notifier {
public:
    void send(const std::string& to, const std::string& msg) override { ... }
};
class SMSNotifier : public Notifier {
public:
    void send(const std::string& num, const std::string& msg) override { ... }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 21 - 같은 역할, 다른 인터페이스
class EmailSender {
    void sendEmail(String to, String body){}
}
class SMSSender {
    void transmit(String num, String msg){} // 이름이 다름
!
}

// ✅ After - 슈퍼클래스/인터페이스 추출
interface Notifier {
    void send(String recipient, String msg);
}
class EmailNotifier implements Notifier {
    public void send(String to, String msg){}
}
class SMSNotifier implements Notifier {
    public void send(String num, String msg){}
}
}
```

22. 뱀새 22. 데이터 클래스 (Data Class)

리팩토링: 인터페이스 통일 · 함수 옮기기 · setter 제거 · 메서드 숨기기

C++ (VSCode / CMake)

```
// ❌ 뱀새 22 - 데이터 클래스 (getter/setter만 있는 클래스)
class Department {
    std::string name_;
    std::vector<Employee> staff_;
    int studentCount_ = 0;
public:
    std::string getName() const { return name_; }
    void setName(const std::string& n) { name_ = n; }
    std::vector<Employee>& getStaff() { return staff_; }
    void setStaff(const std::vector<Employee>& s) { staff_ = s; }
    // 관련 행위가 전혀 없음!
};

// ✅ After - 행위를 클래스로 이동 + setter 제거 + 컬렉션 캡슐화
class Department {
    std::string name_;
    std::vector<Employee> staff_;
public:
    Department(std::string name) : name_(std::move(name)) {}
    std::string getName() const { return name_; } // setter 제거 (불변)
    void addEmployee(Employee e) { staff_.push_back(e); } // 캡슐화
    size_t headCount() const { return staff_.size(); } // 행위 추가
    double totalSalary() const {
        return std::accumulate(staff_.begin(), staff_.end(), 0.0,
            [](double s, const Employee& e){ return s + e.salary; });
    }
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 뱀새 22 - 데이터 클래스 (행위 없음)
class Department {
    private String name;
    private List<Employee> staff;
    String getName(){return name;}
    void setName(String n){name=n;}
    List<Employee> getStaff(){return staff;}
    void setStaff(List<Employee> s){staff=s;}
    // 행위가 전혀 없음!
}

// ✅ After - 행위 이동 + setter 제거
class Department {
    private final String name; // 불변
    private final List<Employee> staff =
        new ArrayList<>();
    Department(String name){this.name=name;}
    void addEmployee(Employee e){staff.add(e);}
    int headCount(){return staff.size();}
    double totalSalary(){
        return staff.stream()
            .mapToDouble(e->e.salary).sum();
    }
}
```

23. 냄새 23. 상속 포기 (Refused Bequest)

리팩토링: 위임으로 교체 · 슈퍼클래스 추출 · 함수 추출 · 어서션 추가

C++ (VSCode / CMake)

```
// ❌ 냄새 23 - 상속 포기: Stack이 vector 기능 모두 상속받는 문제
// std::stack이 vector 상속하는 경우를 상상해보면:
class MyStack : public std::vector<int> {
    // push/pop 기능만 필요한데 vector의 모든 것이 상속됨!
    // insert, erase 등 스택에 부적절한 메서드도 노출
};

// ✅ After - 위임으로 교체 (Replace Inheritance with Delegation)
class MyStack {
    std::vector<int> data_; // 상속 → 위임!
public:
    void push(int val) { data_.push_back(val); }
    int top() const { return data_.back(); }
    void pop() { data_.pop_back(); }
    bool empty() const { return data_.empty(); }
    // vector의 다른 메서드는 노출 안 됨
};

// ✅ Extract Superclass - 공통 부분을 부모로
class Party {
    std::string name_;
public:
    virtual int annualCost() const = 0;
    std::string name() const { return name_; }
};

class Employee : public Party { int annualCost() const override { return salary_; }
};

class Department : public Party { int annualCost() const override {
    int total = 0;
    for (auto& e : staff_) total += e.annualCost();
    return total;
}
};
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 23 - 상속 포기
class Stack extends Vector<Integer> {
    // Vector의 모든 것이 노출!
    // add, remove, indexOf 등 스택에 불필요
}

// ✅ After - 위임으로 교체
class Stack {
    private Vector<Integer> data = new Vector<>();
    void push(int val){ data.add(val); }
    int top() { return data.lastElement(); }
    void pop() { data.remove(data.size()-1); }
    boolean empty() { return data.isEmpty(); }
}

// ✅ Extract Superclass
abstract class Party {
    abstract int annualCost();
    abstract String name();
}

class Employee extends Party{
    int annualCost(){return salary;}
    String name(){return name;}
}
```

리팩토링: 위임으로 교체 · 슈퍼클래스 추출 · 함수 추출 · 어서션 추가

C++ (VSCode / CMake)

```
// ❌ 냄새 24 - 주석이 코드를 설명해야 하는 경우
// 두 조건이 모두 참이어야 배너를 렌더링함 (Mac + IE + 초기화됨 + 크기 변경됨)
if ((platform.find("MAC") != std::string::npos) &&
    (browser.find("IE") != std::string::npos) &&
    wasInitialized && resize > 0) { ... }

// ✅ After - 변수 추출 (Extract Variable) → 주석 불필요
bool isMacOs    = platform.find("MAC") != std::string::npos;
bool isIE       = browser.find("IE") != std::string::npos;
bool wasResized = resize > 0;
if (isMacOs && isIE && wasInitialized && wasResized) { ... }

// ✅ Introduce Assertion - 가정을 코드로
double getExpenseLimit() {
    assert(expenseLimit != NULL_EXPENSE || primaryProject != nullptr);
    return expenseLimit != NULL_EXPENSE ? expenseLimit
                                         : primaryProject->getMemberExpenseLimit();
}
```

Java 대응 (VSCode / Maven)

```
// ❌ 냄새 24 - 주석으로 복잡한 조건 설명
// Mac + IE + 초기화 + 크기변경
if((platform.toUpperCase().indexOf("MAC")>-1)
    &&(browser.toUpperCase().indexOf("IE")>-1)
    && wasInitialized() && resize > 0) {...}

// ✅ After - 변수 추출
boolean isMacOs =
    platform.toUpperCase().contains("MAC");
boolean isIE =
    browser.toUpperCase().contains("IE");
boolean wasResized = resize > 0;
if(isMacOs && isIE
    && wasInitialized() && wasResized){}

// ✅ Introduce Assertion
double getExpenseLimit() {
    assert expenseLimit!=NULL_EXPENSE
        || primaryProject != null;
    return expenseLimit != NULL_EXPENSE
        ? expenseLimit
        : primaryProject.getExpenseLimit();
}
```


VSCode 리팩토링 단축키 & 도구 치트시트 (C++ / Java)

clangd + CMake Tools (C++) | Extension Pack for Java (Java)

★ C++ (clangd) — VSCode 주요 단축키

F2	이름 바꾸기 (Rename Symbol) — 모든 참조 일괄 변경
Ctrl+Shift+R	리팩토링 메뉴 (Extract Function, Move to ...)
Ctrl+.	Quick Fix / Code Action (Extract, Inline, ...)
Ctrl+Shift+P	CMake: Build / CMake: Run Tests 실행
Shift+Alt+F	코드 자동 정렬 (clang-format)
F12 / Alt+F12	정의로 이동 / 미리 보기
Shift+F12	모든 참조 찾기 (Find All References)
Ctrl+Shift+`	터미널 열기 → cmake --build build && ctest

Java — VSCode 주요 단축키

F2	이름 바꾸기 (Rename Symbol)
Ctrl+Shift+R	리팩토링 메뉴 (Extract Method/Class, Move ...)
Ctrl+.	Quick Fix (Extract Method, Convert to Lambda ...)
Ctrl+Shift+P	Java: Run Tests / Maven: Verify 실행
Shift+Alt+F	코드 자동 정렬
F12	정의로 이동
Shift+F12	모든 참조 찾기
Ctrl+Shift+`	터미널 → mvn test -q

필수 VSCode 확장 설치

C++: ms-vscode.cpptools-extension-pack · ms-vscode.cmake-tools · llvm-vs-code-extensions.vscode-clangd

Java: vscjava.vscode-java-pack · redhat.java · vscjava.vscode-maven

Code Smell 24 — 종합 요약표 & 리팩토링 기법 매핑

#	코드 스멜	리팩토링 기법	VSCode 단축키
1	이해하기 힘든 이름	함수/변수/필드 이름 바꾸기	F2 (Rename Symbol)
2	중복 코드	함수 추출하기 / Pull Up Method	Ctrl+Shift+R → Extract
3	긴 함수	함수 추출하기 / 임시변수 질의로	Ctrl+Shift+R → Extract
4	긴 매개변수 목록	매개변수 객체 / 객체 통째로 넘기기	Ctrl+Shift+R → Introduce
5	전역 데이터	변수 캡슐화하기	Ctrl+. → Encapsulate
6	가변 데이터	변수 쪼개기 / 질의변경 함수 분리	Ctrl+. → Split Variable
7	뒤엎긴 변경	단계 쪼개기 / 클래스 추출	Ctrl+Shift+R → Extract Class
8	산탄총 수술	함수/필드 옮기기 / 클래스 인라인	Ctrl+Shift+R → Move
9	기능 편애	함수 옮기기 (Move Function)	Ctrl+Shift+R → Move
10	데이터 뭉치	클래스 추출하기 / 매개변수 객체	Ctrl+Shift+R → Extract Class
11	기본형 집착	기본형→객체 / 타입코드→서브클래스	Ctrl+Shift+R → Extract
12	반복되는 Switch	조건부 로직을 다형성으로	Ctrl+Shift+R → Extract
13	반복문	반복문을 파이프라인으로	Ctrl+. → Convert to Stream
14	성의 없는 요소	함수/클래스 인라인하기	Ctrl+Shift+R → Inline
15	추측성 일반화	죽은 코드 제거하기	Ctrl+Shift+R → Safe Delete
16	임시 필드	클래스 추출 / 특이 케이스 추가	Ctrl+Shift+R → Extract
17	메시지 체인	위임 숨기기	Ctrl+Shift+R → Hide Delegate
18	중재자	중재자 제거 / 위임으로 교체	Ctrl+Shift+R → Remove
19	내부자 거래	함수/필드 옮기기 / 위임 숨기기	Ctrl+Shift+R → Move
20	거대한 클래스	클래스 추출 / 슈퍼클래스 추출	Ctrl+Shift+R → Extract
21	서로 다른 대안 클래스	인터페이스 통일 / 슈퍼클래스 추출	Ctrl+Shift+R → Extract
22	데이터 클래스	Move Method/Field / setter 제거	Ctrl+Shift+R → Move
23	상속 포기	위임으로 교체 / 슈퍼클래스 추출	Ctrl+Shift+R → Extract
24	주석	함수 추출 / 변수 추출 / Assertion	Ctrl+Shift+R → Extract

리팩토링 카탈로그 7가지 분류 — Martin Fowler 2nd Edition

1. 기본 기술

- 함수 추출/인라인
- 변수 추출/인라인
- 함수 선언 변경
- 변수 캡슐화
- 매개변수 객체
- 여러 함수를 클래스로
- 단계 쪼개기

2. 캡슐화

- 레코드 캡슐화
- 컬렉션 캡슐화
- 기본형→객체
- 임시변수→질의
- 클래스 추출/인라인
- 위임 숨기기
- 알고리즘 교체

3. 기능 옮기기

- 함수/필드 옮기기
- 문장을 함수로
- 문장을 호출부로
- 인라인→함수 호출
- 문장 슬라이드
- 반복문 쪼개기
- 죽은 코드 제거

4. 데이터 조직화

- 변수 쪼개기
- 필드 이름 바꾸기
- 파생 변수→질의
- 참조를 값으로
- 값을 참조로

5. 조건부 로직

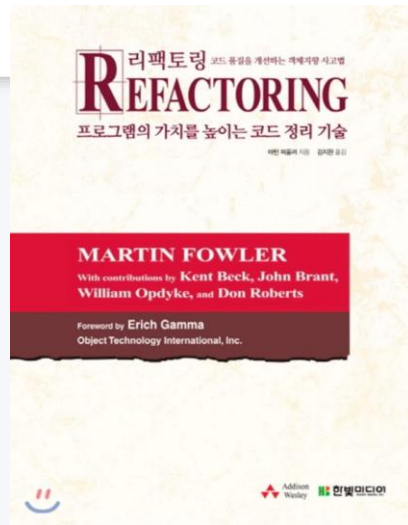
- 조건문 분해
- 조건식 통합
- 중첩 조건→가드
- 조건→다형성
- 특이 케이스 추가
- 어서션 추가

6. API 리팩토링

- 질의/변경 분리
- 함수 매개변수화
- 플러그 인수 제거
- 객체 통째로
- 매개변수→질의
- 세터 제거
- 팩토리 함수

7. 상속 다루기

- 메서드/필드 올리기
- 메서드/필드 내리기
- 타입코드→서브클래스
- 슈퍼클래스 추출
- 계층 합치기
- 서브/슈퍼→위임



리팩토링 관련 참고 동영상

<https://www.youtube.com/watch?v=mNPpfB8JSIU>



Martin Fowler
ThoughtWorks

"Workflows of Refactoring"



실습 파일 제출

프로젝트 명 : RefactoringPractice 제출

TDD 공유 자료실에서 RefactoringPractice-XXX.zip 파일을 다운로드 합니다.



본인 번호를 포함한 Repository 생성 후 구현해 제출하시면 됩니다.

Repository : RefactoringPractice-개인번호 (ex: RefactoringPractice-_01, RefactoringPractice-_02,...)



THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

목표 달성을 위한 여정이 시작됩니다.
궁금한 점이 있으시면 언제든지 문의해주세요!
함께 코더와 프롬프트 전문가로 성장해 나갑시다!

